

# First Run Report

Setup walkthrough and issue-resolution log, from an empty machine to a clean dbt build against real BigQuery data

---

## Olist delivery-performance pipeline

Repository: [github.com/PeterLees/bigData-module2-project-group7](https://github.com/PeterLees/bigData-module2-project-group7)

Companion to: [Module2\\_Group7\\_Development\\_Plan.pdf](#) and [Module2\\_Group7\\_Runbook.pdf](#)

**RESULT: PASS = 182, WARN = 5, ERROR = 0, out of 187 dbt nodes.**

Revenue reconciles to the cent between staging and both fact tables. Two code defects and one mis-scoped test were found and fixed. Five remaining warnings are documented source characteristics, not defects, each with a disposition recorded below.

This report exists because the course guidance treats a fault found, investigated and fixed - with the evidence kept - as stronger proof of a working pipeline than a suite that simply reports green. Everything below is what actually happened on the first real run, not a rehearsed demonstration.

# Contents

---

1. Purpose and How to Read This Report
2. Part 1 - Setup Walkthrough - every step run, and what it does
3. Part 2 - Issue Log - eleven problems found, investigated and fixed
4. Final Verified State
5. Takeaways

# 1. Purpose and How to Read This Report

---

This document has two parts. **Part 1** walks through every step that was actually run to take the project from a bare machine to a clean pipeline execution, in the order it happened, with what each command does and why. **Part 2** is the issue log: every problem encountered along the way, how it was diagnosed, what was changed, and how the fix was verified. Both are written from the real terminal transcript of this run, not reconstructed afterwards.

A short vocabulary, since it recurs constantly below:

| Term                  | Meaning here                                                                                                                                                                                                  |
|-----------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Quality Gate 1</b> | Great Expectations validation of the raw CSV files, run before anything reaches BigQuery. A failure here means nothing loads.                                                                                 |
| <b>Quality Gate 2</b> | dbt build: 24 models, 1 snapshot and (after this run) 162 tests, run in dependency order. A test at error severity blocks the marts from publishing; a test at warn severity publishes and reports the count. |
| <b>Reconciliation</b> | An assertion that a total computed one way (e.g. in staging) equals the same total computed another way (e.g. in a mart). Used throughout this project to prove no join silently multiplied money.            |
| <b>Disposition</b>    | The decision made about a finding: fix the code, fix the test, or accept and document as a known characteristic of the source data at warn severity.                                                          |

## 2. Part 1 - Setup Walkthrough

---

Steps 2.1 to 2.5 below are the one-time setup described in the Runbook. They are repeated here with the actual commands run and their real output, because several of them surfaced problems that are easiest to understand in sequence.

### 2.1 Python environment

The project pins Python 3.11, because dbt does not support the machine's default 3.14. The setup step is:

```
conda create -y -n bd-m2-g7 python=3.11
conda activate bd-m2-g7
make setup      # pip install -r requirements.txt; dbt deps
```

`make setup` failed on its first real invocation with a pip dependency conflict, and separately the team member had activated the wrong conda environment when running it. Both are full issues in Part 2 (Issues 1 and 2); the corrected sequence above already reflects the fix.

### 2.2 GCP project

A project named `bigdata-m2-group7` was created in the Google Cloud console, billing was linked (the always-free tier covers this project's ~120 MB of data many times over), and two APIs were enabled one at a time: the **BigQuery API** first, confirmed via its status page showing Enabled and a Disable API button, then the **Cloud Storage API** once it was noticed the first check only covered one of the two required APIs. The team was invited with the BigQuery Job User role.

Verification used throughout the project rather than trusting the console screenshot alone: `gcloud services list --enabled --filter="bigquery.googleapis.com OR storage.googleapis.com"` and `gcloud config get-value project` to confirm the active project ID matches `.env`.

### 2.3 Local authentication

```
gcloud auth login
gcloud auth application-default login
gcloud config set project bigdata-m2-group7
```

The second command is the one most often skipped: it creates the Application Default Credentials that dbt, the Python BigQuery client, SQLAlchemy and the Streamlit dashboard all read, and it is a separate credential from the first command. Both were run and later confirmed present at `~/config/gcloud/application_default_credentials.json`.

### 2.4 Dataset download

The Olist CSV export was downloaded from Kaggle and unzipped into `data/raw/`. A question was raised about whether the data also needed to be uploaded to BigQuery through the console - it does not, and should not be: doing so would bypass Quality Gate 1, the explicit schemas, the manifest and the row-count reconciliation that `make ingest` provides, and would break the idempotency guarantee the project is graded on. The CSVs are read directly by the ingestion script; no manual upload step exists in this pipeline.

## 2.5 Bucket, datasets and configuration

```
gcloud storage buckets create gs://bigdata-m2-group7-olist-raw \
  --location=US --uniform-bucket-level-access

for ds in olist_raw olist_staging olist_marts olist_snapshots; do
  bq --location=US mk --dataset "bigdata-m2-group7:$ds"
done

cp .env.example .env          # then fill in GCP_PROJECT_ID, GCS_RAW_BUCKET, DBT_DEV_DATASET
mkdir -p ~/.dbt && cp dbt_project/profiles.yml.example ~/.dbt/profiles.yml
```

The bucket-creation command was pasted twice into the terminal in one go, which produced a parse error rather than a clean failure (Issue 3 in Part 2). Once resolved, `.env` was populated with `GCP_PROJECT_ID=bigdata-m2-group7`, `GCS_RAW_BUCKET=bigdata-m2-group7-olist-raw` and `DBT_DEV_DATASET=dbt_peter`, and the four shared BigQuery datasets plus one personal development dataset were created, all pinned to US multi-region to match the project's locked architecture decision.

## 2.6 Connectivity check

```
make debug
```

This is the step that should confirm every piece above is wired together correctly: gcloud authentication, `.env`, and the dbt profile. It failed on its first invocation for a reason that took some tracing to pin down (Issue 4), and a related Makefile defect was found while fixing it (Issue 5). Once both were fixed, the check passed cleanly:

```
method: oauth
database: bigdata-m2-group7
schema: dbt_peter
location: US
Connection test: [OK connection ok]

All checks passed!
```

## 2.7 First pipeline run

```
make all          # manifest -> Quality Gate 1 -> ingest -> Quality Gate 2 (dbt build) -> pytest
```

This is where the real work happened. Quality Gate 1 failed on its first pass over the geolocation file (Issue 6). Once the pipeline could load data, `dbt build` ran for the first time against a populated warehouse and reported six errors (Issues 7 through 11, several of them variants of the same underlying finding). Each is documented in full in Part 2. The final, clean run is summarised in Section 4.

## 3. Part 2 - Issue Log

Eleven items, in the order they were encountered. Each card states what tag caused it: **code** (a bug in project source), **test** (a test whose logic or thresholds were wrong, not the model it was testing), **config** (a Makefile or requirements defect), or **human** (an operational slip with no code change required). For every code and test fix, the underlying data was queried directly against BigQuery before writing the fix, so each disposition below is backed by a number, not a guess.

### Issue 1 - pip dependency conflict blocked `make setup`

**CONFIG**

#### SYMPTOM

ResolutionImpossible: streamlit 1.62.0 requires altair>=5.0.0,<7, while great-expectations 1.8.0 requires altair<5.0.0,>=4.2.1 - mutually exclusive. A second, hidden conflict surfaced once the first was fixed: dagster-dbt 0.28.8 caps dbt-core below 1.11, clashing with the pinned dbt-core==1.11.11.

#### INVESTIGATION

Used `pip install --dry-run --report` against PyPI to let pip's own resolver find a working version set, rather than guessing pins by hand. Confirmed Great Expectations dropped the `altair<5` constraint after 1.8.0.

#### FIX

great-expectations 1.8.0 → **1.22.0**; dagster/dagster-dbt/dagster-webserver 1.12.8/0.28.8/1.12.8 → **1.13.20/0.29.20/1.13.20**; removed the now-redundant `sqlfluff-templater-dbt` package (linting is configured to use the jinja templater, which needs no warehouse profile).

#### VERIFICATION

Resolved the full `requirements.txt` cleanly (220 packages). Re-ran the Great Expectations fault-injection fixture on 1.22.0: correctly PASS on clean data, correctly FAIL with the right column and count on injected faults. Loaded the Dagster 1.13.20 definitions and confirmed 28 assets, 1 job, 1 daily schedule - unchanged from the pre-upgrade baseline.

### Issue 2 - wrong conda environment activated

**HUMAN**

#### SYMPTOM

`make setup` was run from `conda activate bde`, the pre-existing course environment (221 packages, including dagster 1.12.8), rather than the project's own `bd-m2-g7` environment.

#### INVESTIGATION

Compared both environments directly: `bd-m2-g7` already existed - Python 3.11.15, 4 packages, untouched - while `bde` was a shared course environment that other coursework may depend on. Because Issue 1's conflict made pip fail before installing anything, no packages were actually changed in `bde`.

#### FIX

No code change. Directed the team to `conda activate bd-m2-g7` before every project command.

#### VERIFICATION

Re-ran `pip list` in `bde` afterward and confirmed it was unaffected.

### Issue 3 - bucket-creation command produced a parse error

HUMAN

#### SYMPTOM

`gcloud storage buckets create ...` was pasted twice in one terminal action. The trailing backslash of the first copy swallowed the entire second copy as a single positional argument, producing `Received: "file://gcloud"`.

#### INVESTIGATION

The output showed `Creating gs://bigdata-m2-group7-olist-raw/...` before the error, meaning the bucket may already have been created successfully before the malformed second invocation was parsed.

#### FIX

No code change. Verified with `gcloud storage ls` whether the bucket already existed before re-running; re-ran the single-line command otherwise. Separately flagged that the bucket name used, `bigdata-m2-group7-olist-raw`, has no numeric suffix even though the project ID was expected to (Google does not always append one), and confirmed `.env`'s `GCS_RAW_BUCKET` matches the bucket that actually exists rather than the plan's template name.

#### VERIFICATION

`gcloud storage ls` confirmed exactly one bucket, matching `.env`.

### Issue 4 - bare `dbt debug` could never see `.env`

CONFIG

#### SYMPTOM

`cd dbt_project && dbt debug` failed with `Env var required but not provided: 'GCP_PROJECT_ID'`, despite `.env` being present and correctly filled in.

#### INVESTIGATION

Confirmed `.env` held the right values and that `gcloud config get-value project` matched an ACTIVE project. The Makefile is the *only* place that loads and exports `.env` (via `include .env / export`, added earlier in the project specifically to fix this class of problem for every other target). A bare `dbt` invocation run outside `make` bypasses that entirely and `dbt's env_var()` sees nothing.

#### FIX

Added a dedicated `make debug` target (`$(DBT) debug --target $(DBT_TARGET)`) so the connection check can no longer be invoked without the environment being loaded. Corrected the instruction to `make debug` everywhere it appeared: the Development Plan PDF, the Runbook PDF, and `notebooks/00_runbook.ipynb`.

#### VERIFICATION

`make debug` now reports `Connection test: [OK connection ok]` and `All checks passed!` against the real project.

## Issue 5 - make help printed "Makefile" as every target name

[CONFIG](#)

### SYMPTOM

Found while fixing Issue 4. `make help` listed every target's name as the literal word `Makefile` instead of e.g. `setup` or `gate`.

### INVESTIGATION

The `include .env` line added for Issue 4's fix appends `.env` to Make's built-in `MAKEFILE_LIST` variable. The `help` target's `grep` scans `$(MAKEFILE_LIST)`, and `grep` across two files prefixes every match with its source filename; the downstream `awk` then read that filename as the target name.

### FIX

Changed the `help` target to scan only `$(firstword $(MAKEFILE_LIST))` - the actual `Makefile` - with `grep -h` as a second safeguard against the filename prefix.

### VERIFICATION

`make help` now lists all seventeen targets with their real names and descriptions.

## Issue 6 - Quality Gate 1 blocked the load on geolocation

[TEST](#)

### SYMPTOM

On the first real `make gate`, 8 of 9 tables passed; `geolocation` failed with 29 values outside the latitude bound and 26 outside the longitude bound.

### INVESTIGATION

Queried the 29 offending rows directly rather than assuming corruption. They carry genuine Brazilian city and state names (e.g. *vila nova de campos, RJ*; *santa lucia do piai, RS*) but coordinates that place them in Spain, Mexico, Italy, the Philippines and Argentina - **not** the Indian Ocean, which is what swapped latitude/longitude columns would produce. This is 0.0029% of 1,000,163 rows: a documented geocoding-noise characteristic of the Olist export, not a structural error.

### FIX

Changed the Great Expectations rule from an absolute bound to a **threshold** (`mostly=0.999`). This still fails hard on a wrong-country export or swapped columns - either would blow through the threshold by orders of magnitude - while tolerating the known 29 outliers. Recorded as data issue 13 in `docs/data_dictionary.md`, and noted that `stg_geolocation` already collapses points to a per-zip-prefix **median** rather than a mean, specifically so isolated bad points cannot move a whole prefix.

### VERIFICATION

Re-ran the gate: 9/9 tables PASS. Also quieted Great Expectations' verbose INFO logging (it was emitting roughly a dozen lines per table and burying the PASS/FAIL result), setting its logger to WARNING.



## Issue 7 - stg\_geolocation failed to compile

[CODE](#)

### SYMPTOM

First dbt build against real data: Window ORDER BY expression references column geolocation\_city which is neither grouped nor aggregated. This blocked 65 downstream models via SKIP.

### INVESTIGATION

The model's QUALIFY row\_number() OVER (...) clause ordered by geolocation\_city, the raw source column - but QUALIFY is evaluated after SELECT, by which point only the grouped aliases (city, zip\_code\_prefix) exist.

### FIX

Changed the QUALIFY clause to reference zip\_code\_prefix and city instead of the raw column names, with a comment explaining why the raw names cannot work at that point in the query.

### VERIFICATION

stg\_geolocation compiled and ran (view created in ~3 seconds); all 65 previously-skipped downstream models became eligible to build.

## Issue 8 - milestone-ordering test failed on 1,382 rows

[TEST](#)

### SYMPTOM

assert\_delivery\_milestones\_in\_order returned 1,382 failing rows, configured to fail on any non-zero result.

### INVESTIGATION

Broke the 1,382 down by which pair of milestones was out of order: approved\_before\_purchase = 0, delivered\_before\_purchase = 0 (the two orderings that are physically impossible), against carrier\_handover\_before\_approval = 1,359 and delivered\_before\_carrier = 23 (the two that are merely unusual). Measured the magnitude of the latter two: a median gap of 21 hours and roughly a 4-day p95 for the carrier/approval case. That pattern - hours to a few days, never negative in the impossible direction - is the signature of **asynchronous event recording**: a Brazilian boleto payment settles over one to three business days, so a seller can visibly hand a parcel to the carrier before the payment-approval row is written, not because the order of events was wrong but because the two systems that record them are not synchronized.

### FIX

Split one test into two. assert\_delivery\_milestones\_in\_order keeps error severity but now checks only the physically-impossible orderings (0 rows). A new test, assert\_milestone\_recording\_order, checks the asynchronous-recording cases at warn severity, with the investigation above recorded in its header comment. The rationale for not blocking the build: total\_delivery\_days and delay\_vs\_promise\_days, the two measures the project's headline late-rate figure is built from, are both computed from order\_purchased\_at, which is never affected. Only the stage-level decomposition (seller\_handover\_days) is affected, and only for 1.4% of orders.

### VERIFICATION

The impossible-ordering test now passes with 0 rows on every run; the new warn-severity test reports the 1,382-row count so it can be monitored for growth.

## Issue 9 - 14 orders with a status/timestamp mismatch

TEST

### SYMPTOM

`assert_delivered_orders_have_delivery_date` returned 14 failing rows at error severity.

### INVESTIGATION

Split the 14 by direction: 8 orders carry status `delivered` but no delivery timestamp; 6 carry status `canceled` but do have a delivery timestamp (delivered, then refunded or cancelled afterward - a normal business outcome, not a data defect). The 8 delivered-without-timestamp orders were already handled correctly by design elsewhere in the pipeline: `is_late` is NULL for them, which removes them from the late-rate denominator rather than silently counting them as on time.

### FIX

Changed the test to warn severity. Blocking the entire build on 0.014% of orders whose already-correct handling was being re-litigated by an over-strict test was judged the wrong response; the count is retained as a monitored figure instead.

### VERIFICATION

The test now reports Got 14 results, configured to warn if `!= 0` without stopping the build, and the two sub-cases are documented in its header comment for the next person who reads it.

## Issue 10 - three delivery-duration range tests failed on real bounds

TEST

### SYMPTOM

Three `dbt_utils.accepted_range` tests failed: `approval_days` (2 rows outside `[-1, 60]`), `seller_handover_days` (28 rows outside `[-5, 180]`), `carrier_transit_days` (6 rows outside `[-5, 210]`). These bounds had been set by estimate before any real data existed.

### INVESTIGATION

Queried the actual min/max of each column against the loaded data: `approval_days` spans 0.0 to 187.9 (confirmed zero rows below 0 - the floor is a genuine constraint, since approval cannot precede purchase); `seller_handover_days` spans -171.2 to 125.8 (the negative tail is exactly the Issue 8 recording artefact); `carrier_transit_days` spans -16.1 to 205.2 (same artefact).

### FIX

Recalibrated each bound from the observed distribution, with headroom sized to still catch a genuine parsing or timezone corruption rather than to just pass the current data: `approval_days` to `[0, 365]`, `seller_handover_days` to `[-200, 365]`, `carrier_transit_days` to `[-30, 365]`. Each change is commented with the observed range and a cross-reference to `assert_milestone_recording_order` for why negative values are expected.

### VERIFICATION

All three tests pass on the current data while remaining tight enough to fail on an order-of-magnitude error.

**SYMPTOM**

After Issues 7-10 were fixed, the rebuild reported PASS=181 WARN=5 ERROR=1 - one failure remained:

assert\_rfm\_uses\_person\_grain, comparing agg\_customer\_rfm's row count (94,990) against all of dim\_customer (96,096).

**INVESTIGATION**

Queried the gap directly: the number of distinct people holding at least one order that survives the excluded-status filter (canceled/unavailable) is exactly 94,990 - matching the RFM row count precisely. The other 1,106 people in dim\_customer placed *only* cancelled or unavailable orders, for whom no recency, frequency or monetary value can be computed. Excluding them from RFM is correct behaviour; the test's comparison, not the model, was wrong.

**FIX**

Rewrote the test to compare agg\_customer\_rfm against the count of people with a *kept* order rather than against all of dim\_customer, and strengthened it by also asserting the RFM row count stays materially below the distinct *source* customer\_id count (99,441) - a direct, numeric check for the exact trap this test exists to catch, since customer\_id is issued per order rather than per person.

**VERIFICATION**

The test now passes, and the rebuild reports ERROR=0.

## 4. Final Verified State

### 4.1 dbt build result

|         |                                                                                                                                             |
|---------|---------------------------------------------------------------------------------------------------------------------------------------------|
| 1st run | manifest and Quality Gate 1 pass; ingest and reconciliation pass. Not yet attempted: dbt build.                                             |
| 2nd run | <b>dbt build, before any fix.</b> PASS=115 WARN=0 ERROR=6 SKIP=65 - Issue 7 (a compile error) skipped 65 downstream models outright.        |
| 3rd run | <b>dbt build, after Issues 7-10.</b> PASS=181 WARN=5 ERROR=1 - only the RFM test (Issue 11) remained.                                       |
| 4th run | <b>dbt build, after Issue 11.</b> PASS=182 WARN=5 ERROR=0, out of 187 total nodes (1 snapshot, 12 table models, 12 view models, 162 tests). |

### 4.2 Reconciliation - the test that matters most

Total revenue, computed three independent ways, agrees to the cent:

| Source                                        | Value (BRL)   |
|-----------------------------------------------|---------------|
| stg_order_items (staging)                     | 13,591,643.70 |
| fct_order_items (item-grain fact)             | 13,591,643.70 |
| fct_orders (order-grain fact, pre-aggregated) | 13,591,643.70 |

Order counts likewise agree exactly: 99,441 distinct orders in staging, 99,441 rows in fct\_orders. No join anywhere in the DAG inflated either figure.

### 4.3 First measured business numbers

These are the first real values the delivery-performance business case has produced, computed from the live warehouse rather than from synthetic data used during earlier development.

|                                    |                            |                         |                                 |                              |                                          |
|------------------------------------|----------------------------|-------------------------|---------------------------------|------------------------------|------------------------------------------|
| <b>6.77%</b><br>LATE-DELIVERY RATE | <b>10.2</b><br>MEDIAN DAYS | <b>23.1</b><br>P90 DAYS | <b>4.29 ★</b><br>SCORE, ON TIME | <b>2.27 ★</b><br>SCORE, LATE | <b>R\$986K</b><br>REVENUE DELIVERED LATE |
|------------------------------------|----------------------------|-------------------------|---------------------------------|------------------------------|------------------------------------------|

96,470 delivered orders were in scope. A late delivery costs, on average, roughly **two full stars** of review score - the delivery-to-satisfaction link the project's business case was designed around is real and large in this data, not just plausible in theory.

### 4.4 Warnings carried forward, with disposition

| Warning                                    | Count | Disposition                                                                                                            |
|--------------------------------------------|-------|------------------------------------------------------------------------------------------------------------------------|
| assert_milestone_recording_order           | 1,382 | Asynchronous event recording (Issue 8); headline metrics unaffected.                                                   |
| assert_payment_total_matches_order_value   | 296   | Voucher payments legitimately reduce the amount charged versus merchandise + freight.                                  |
| assert_delivered_orders_have_delivery_date | 14    | 8 delivered-without-timestamp (already correctly excluded via NULL is_late) plus 6 delivered-then-cancelled (Issue 9). |

| Warning                         | Count      | Disposition                                                  |
|---------------------------------|------------|--------------------------------------------------------------|
| stg_geolocation latitude range  | 5 prefixes | Mis-geocoded points surviving the median collapse (Issue 6). |
| stg_geolocation longitude range | 6 prefixes | Same source characteristic.                                  |

All five are documented in docs/data\_dictionary.md under "First real BigQuery run - findings and dispositions", along with the two code defects and the mis-scoped test. Run artifacts (run\_results.json, dbt\_manifest.json, the ingestion manifest) are archived in docs/evidence/.

## 5. Takeaways

---

- **The two quality gates did exactly what they were built to do.** Gate 1 stopped a load over 0.003% of mis-geocoded rows and forced a decision about whether that mattered (it did not, once investigated). Gate 2 stopped a mart from publishing over a genuine SQL bug (Issue 7), which is precisely the failure mode the gate exists to catch before it reaches a chart.
- **Every fix in this log was preceded by a query against the real data**, not a guess about what the data probably looked like. The bounds in Issue 10 and the disposition in Issues 6, 8, 9 and 11 are only correct because the actual distribution and the actual row counts were checked first.
- **Not every failing test meant broken data.** Four of the six original errors (Issues 8, 9, 10 in part, and the geolocation gate in Issue 6) were tests whose thresholds or scope were wrong, not defects in the models they were testing. Distinguishing "the code is wrong" from "the test is wrong" required looking at the actual numbers in both directions before writing a fix.
- **The Makefile-centralised `.env` loading (Issue 4) is a real trade-off**, not a free win: it makes every `make` target consistent, but it means any command run outside `make` silently loses the environment. That trade-off is now documented at the point where it bites (the Runbook), not just in the Makefile's own comments.
- **The reconciliation tests are the ones worth trusting most.** Every other test in this log had a legitimate reason to be relaxed to `warn`. The revenue and order-count reconciliations did not, and they passed on the very first attempt once the SQL bug was fixed - which is the strongest available evidence that no join in this pipeline silently multiplies money.